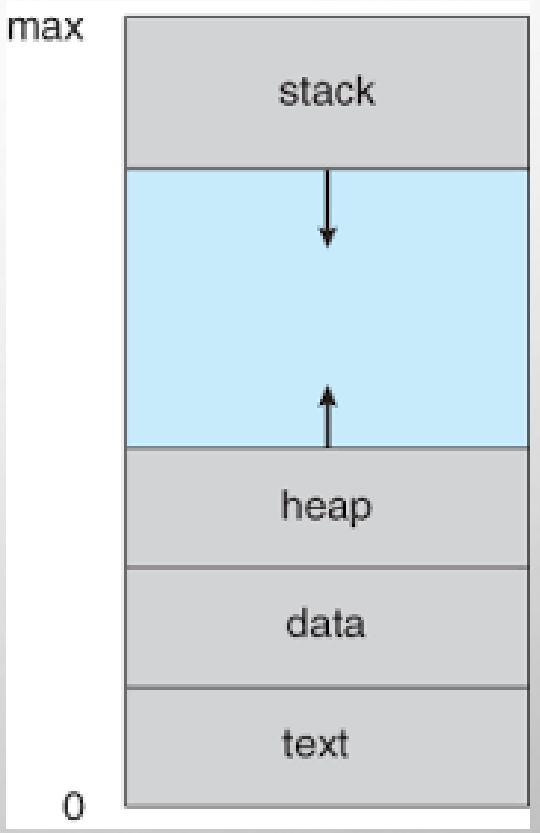
CHAPTER 3: PROCESSES

PROCESS

- A **PROCESS** is basically a **program that's currently being executed —** not just sitting on disk, but *alive and running*
- **Program** → a passive file (like chrome.exe stored on your disk)
- **PROCESS** → an active instance of that program in memory (chrome actually running)
- You can even have **multiple processes of the same program** running at once.

SECTIONS OF PROCESS AND THEIR MEMORY LOCATION

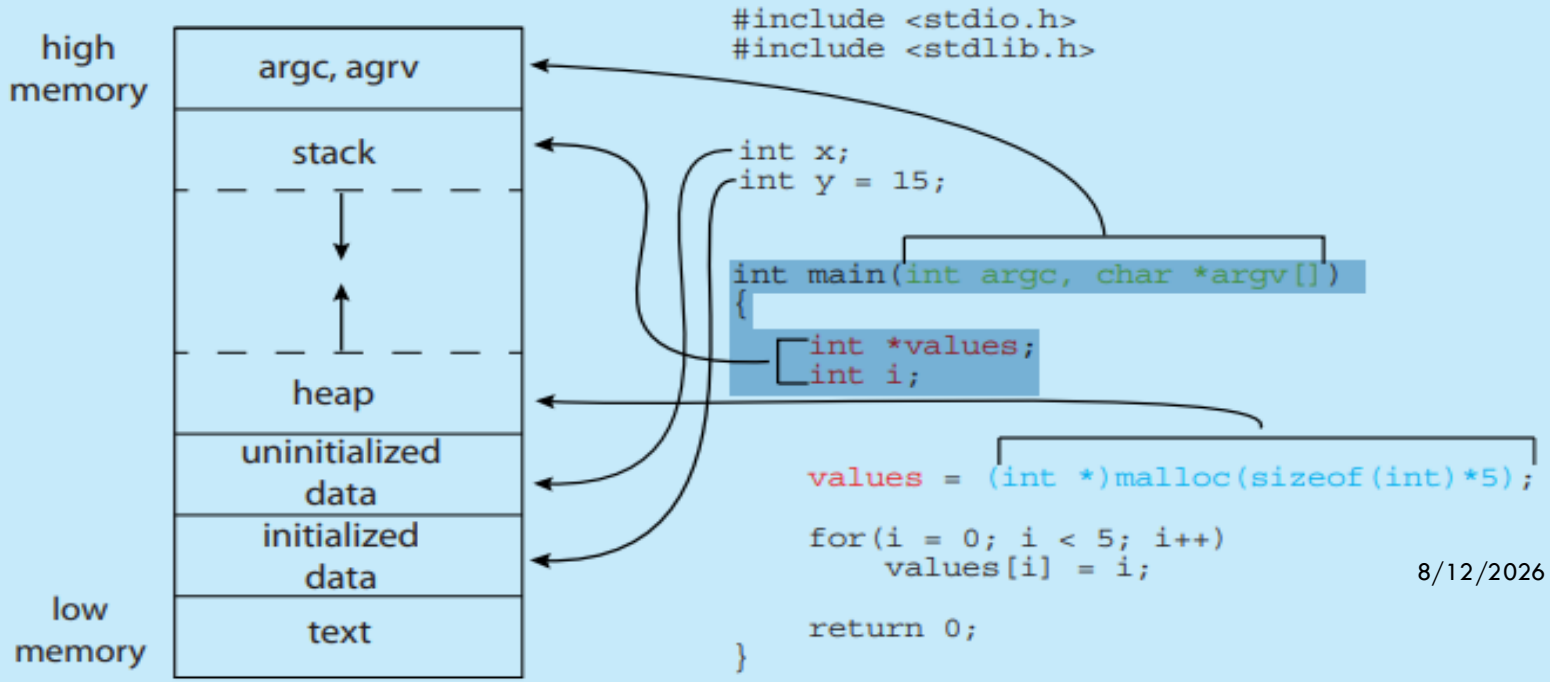
- **TEXT SECTION:** THE EXECUTABLE CODE
- **DATA SECTION:** GLOBAL VARIABLES
- **HEAP:** MEMORY THAT IS DYNAMICALLY ALLOCATED DURING PROGRAM RUN TIME
- **STACK:** CONTAINING TEMPORARY DATA, FUNCTION PARAMETERS, RETURN ADDRESSES, LOCAL VARIABLES, DATA SECTION CONTAINING GLOBAL



MEMORY LAYOUT OF A C PROGRAM

The figure shown below illustrates the layout of a C program in memory, highlighting how the different sections of a process relate to an actual C program. This figure is similar to the general concept of a process in memory as shown in Figure 3.1, with a few differences:

- The global data section is divided into different sections for (a) initialized data and (b) uninitialized data.
- A separate section is provided for the `argc` and `argv` parameters passed to the `main()` function.

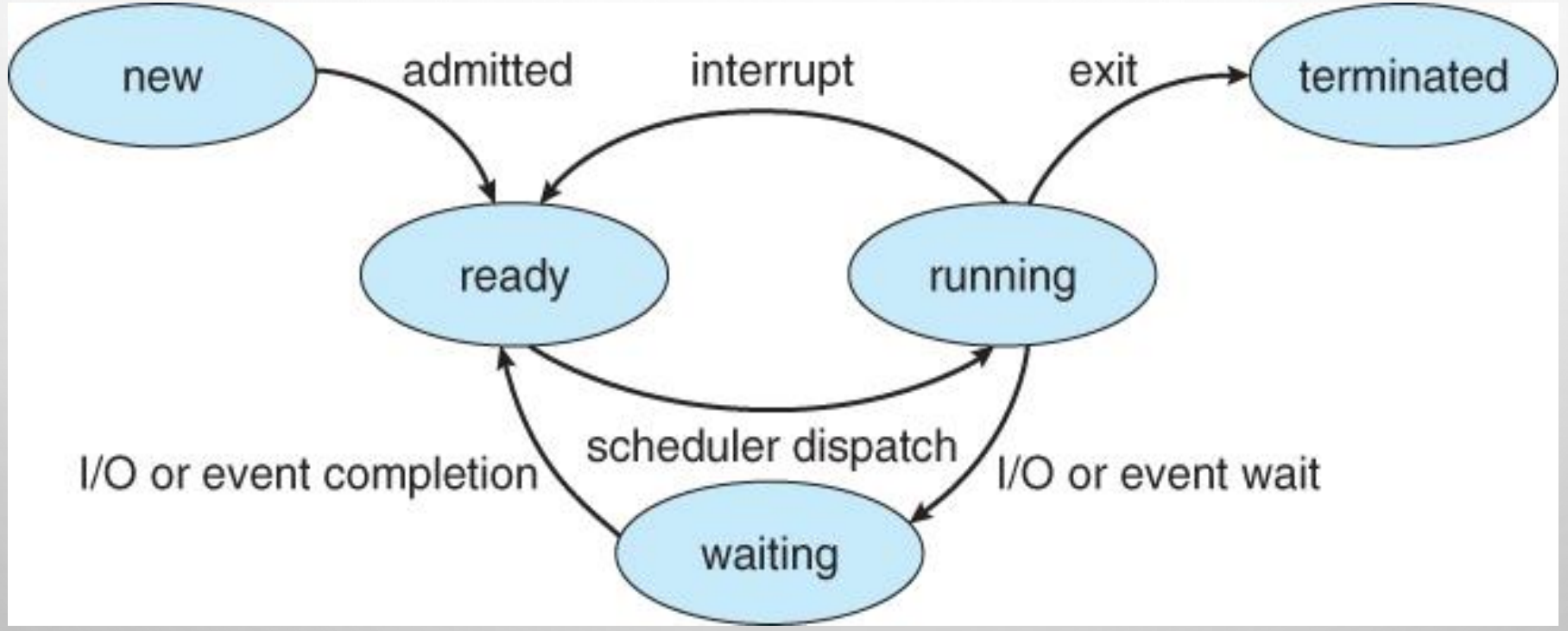


PROCESS STATE

AS A PROCESS EXECUTES, IT CHANGES STATE

- **NEW**: THE PROCESS IS BEING CREATED
- **RUNNING**: INSTRUCTIONS ARE BEING EXECUTED
- **WAITING**: THE PROCESS IS WAITING FOR SOME EVENT TO OCCUR
- **READY**: THE PROCESS IS WAITING TO BE ASSIGNED TO A PROCESSOR
- **TERMINATED**: THE PROCESS HAS FINISHED EXECUTION

DIAGRAM OF PROCESS STATE



PROCESS CONTROL BLOCK(CONT'D)

Each process is represented in the operating system by a process control block (PCB)—also called a task control block. It contains many pieces of information associated with a specific process, including these:

- **PROCESS STATE.** THE STATE MAY BE NEW, READY, RUNNING, WAITING, HALTED, AND SO ON.
- **PROGRAM COUNTER.** THE COUNTER INDICATES THE ADDRESS OF THE NEXT INSTRUCTION TO BE EXECUTED FOR THIS PROCESS.
- **CPU REGISTERS.** THE REGISTERS VARY IN NUMBER AND TYPE, DEPENDING ON THE COMPUTER ARCHITECTURE. THEY INCLUDE ACCUMULATORS, INDEX REGISTERS, STACK POINTERS, AND GENERAL-PURPOSE REGISTERS, PLUS ANY CONDITION-CODE INFORMATION. ALONG WITH THE PROGRAM COUNTER, THIS STATE INFORMATION MUST BE SAVED WHEN AN INTERRUPT OCCURS, TO ALLOW THE PROCESS TO BE CONTINUED CORRECTLY AFTERWARD WHEN IT IS RESCHEDULED TO RUN.

PROCESS CONTROL BLOCK(CONT'D)

- **CPU-SCHEDULING INFORMATION.** THIS INFORMATION INCLUDES A PROCESS PRIORITY, POINTERS TO SCHEDULING QUEUES, AND ANY OTHER SCHEDULING PARAMETERS. (CHAPTER 5 DESCRIBES PROCESS SCHEDULING.)

- **MEMORY-MANAGEMENT INFORMATION**. THIS INFORMATION MAY INCLUDE SUCH ITEMS AS THE VALUE OF THE BASE AND LIMIT REGISTERS AND THE PAGE TABLES, OR THE SEGMENT TABLES, DEPENDING ON THE MEMORY SYSTEM USED BY THE OPERATING SYSTEM (CHAPTER 9).

- **ACCOUNTING INFORMATION**. THIS INFORMATION INCLUDES THE AMOUNT OF CPU AND REAL TIME USED, TIME LIMITS, ACCOUNT NUMBERS, JOB OR PROCESS NUMBERS, AND SO ON.

- **I/O STATUS INFORMATION**. THIS INFORMATION INCLUDES THE LIST OF I/O DEVICES ALLOCATED TO THE PROCESS, A LIST OF OPEN FILES, AND SO ON.

PROCESS AND THREAD

- A **PROCESS** IS AN INDEPENDENT PROGRAM IN EXECUTION
- A **THREAD** IS A LIGHTWEIGHT UNIT OF EXECUTION INSIDE A PROCESS(A PROCESS MAY HAVE MULTIPLE THREADS.)

EASY ANALOGY

PROCESS → A HOUSE

THREAD → ROOMS INSIDE THE HOUSE (SHARE ELECTRICITY, WATER)

PROCESS AND THREAD

Basis	Process	Thread
Definition	A **process** is an independent program in execution	A **thread** is a lightweight unit of execution inside a process
Memory	Has **separate memory space**	**Shares memory** with other threads of the same process
Creation	**Heavyweight** (takes more time/resources)	**Lightweight** (faster to create)
Communication	Uses **IPC** (pipes, sockets, etc.)	Direct communication via **shared memory**
Overhead	High overhead	Low overhead
Isolation	Strongly isolated (safe from other processes)	Not isolated (one thread can affect others)
Context Switching	Slower	Faster
Failure Impact	Crash affects only that process	Crash can terminate the whole process
Example	Running Chrome, Word, VLC	Multiple tabs in Chrome

PROCESS SCHEDULING (CONT'D)

- **PROCESS SCHEDULING** IS THE METHOD AN OPERATING SYSTEM USES TO DECIDE WHICH PROCESS (PROGRAM IN EXECUTION) GETS TO RUN WHEN MULTIPLE PROCESSES ARE READY. THE PROCESS SCHEDULER SELECTS A PROCESS FROM THE *READY QUEUE* AND ALLOCATES CPU TIME TO IT.

IT MAINTAINS SCHEDULING QUEUES OF PROCESSES

- **JOB QUEUE** – SET OF ALL PROCESSES IN THE SYSTEM
  - Stored in secondary storage (disk)
  - Managed by the long-term scheduler
  - Not all processes in this queue are ready to run
- **DEVICE QUEUE**

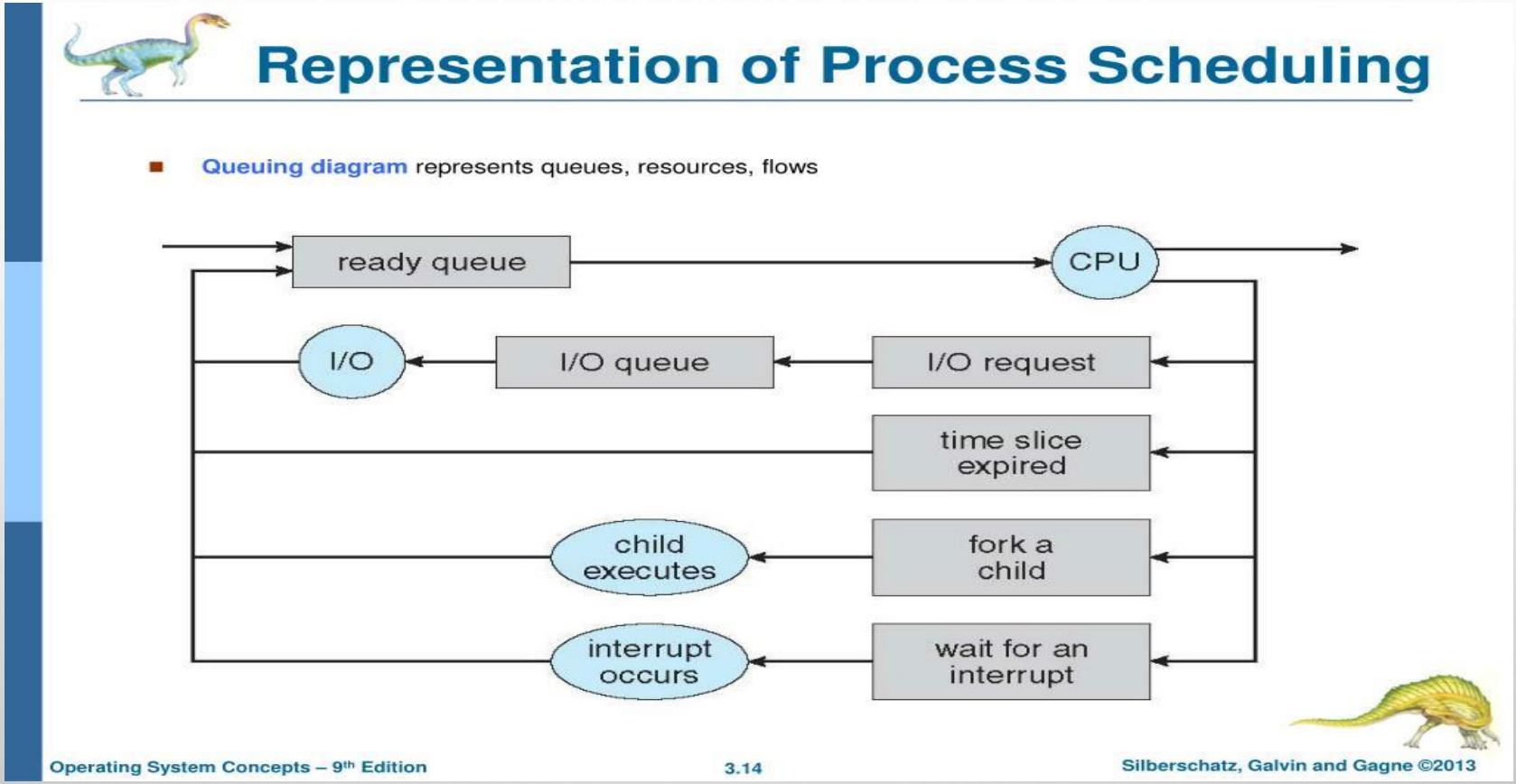
EACH I/O DEVICE MAY HAVE ITS OWN QUEUE. EXAMPLE: PRINTER QUEUE, DISK QUEUE

PROCESS SCHEDULING (CONT'D)

- **READY QUEUE**
  THE **READY QUEUE** CONTAINS PROCESSES THAT ARE IN MAIN MEMORY AND READY TO EXECUTE, WAITING FOR CPU TIME.
  - stored in main memory (RAM)
  - processes are in ready state
  - managed by the short-term scheduler
  - CPU selects processes from here
- **WAITING QUEUE**

CONTAINS PROCESSES THAT ARE WAITING FOR I/O OPERATIONS (LIKE KEYBOARD INPUT, FILE READ, PRINTER).

PROCESS SCHEDULING (CONT'D)



Queueing-diagram representation of process scheduling.

SCHEDULERS

SHORT-TERM SCHEDULER (OR CPU SCHEDULER) – SELECTS WHICH PROCESS SHOULD BE EXECUTED NEXT AND ALLOCATES CPU

- ❑ SOMETIMES THE ONLY SCHEDULER IN A SYSTEM
- ❑ SHORT-TERM SCHEDULER IS INVOKED FREQUENTLY (MILLISECONDS) (MUST BE FAST)

LONG-TERM SCHEDULER (OR JOB SCHEDULER) – SELECTS WHICH PROCESSES SHOULD BE BROUGHT INTO THE READY QUEUE

- ❑ LONG-TERM SCHEDULER IS INVOKED INFREQUENTLY (SECONDS, MINUTES) (MAY BE SLOW)
- ❑ THE LONG-TERM SCHEDULER CONTROLS THE DEGREE OF MULTIPROGRAMMING

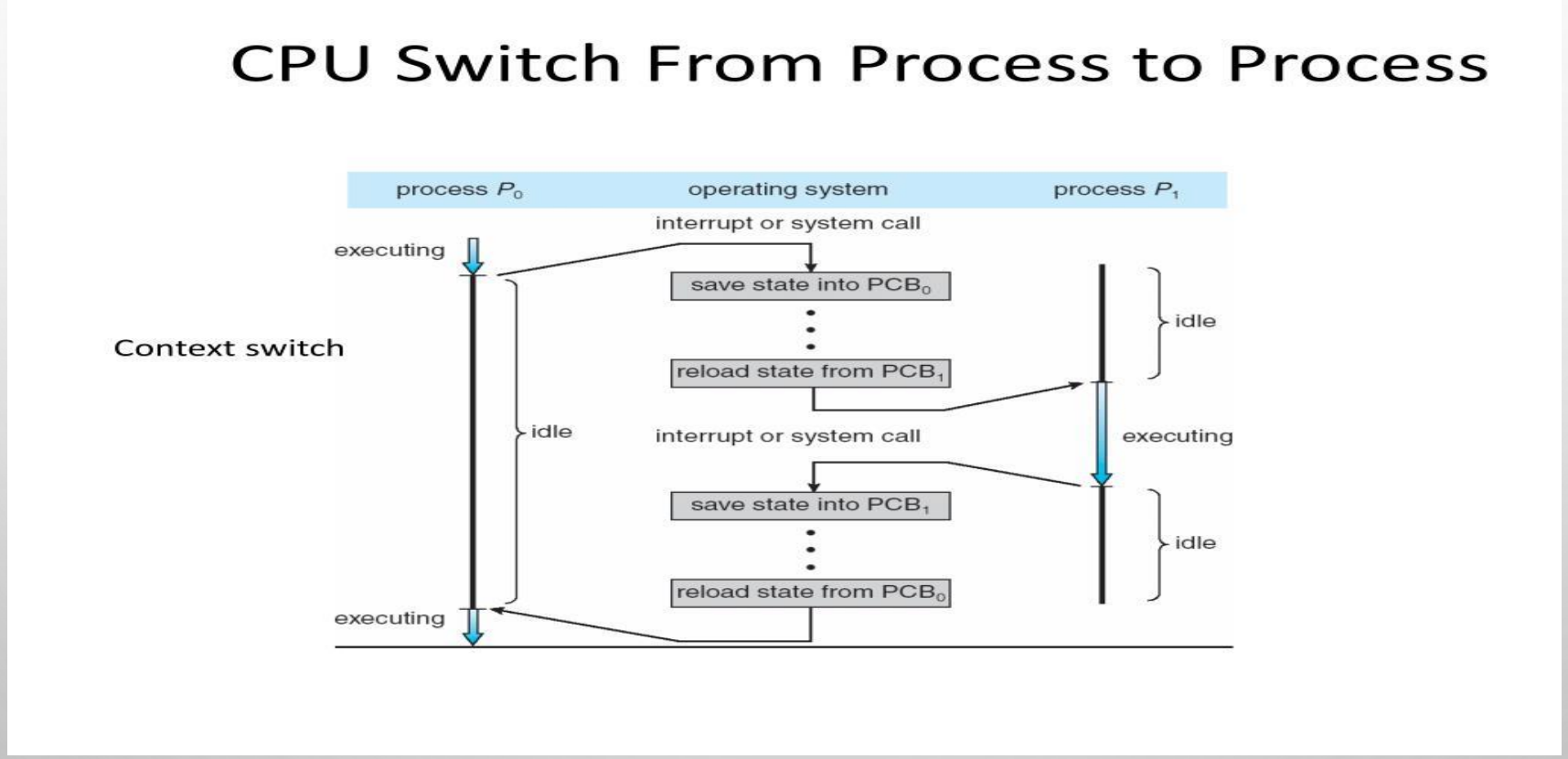
➢ PROCESSES CAN BE DESCRIBED AS EITHER: I/O-BOUND PROCESS– SPENDS MORE TIME DOING I/O THAN COMPUTATIONS, MANY SHORT CPU BURSTS .CPU-BOUND PROCESS – SPENDS MORE TIME DOING COMPUTATIONS; FEW VERY LONG CPU BURSTS

CONTEXT SWITCHING

- **CONTEXT SWITCHING** IS THE OPERATION BY WHICH AN OPERATING SYSTEM SAVES THE EXECUTION STATE (CONTEXT) OF A CURRENTLY RUNNING PROCESS OR THREAD AND RESTORES THE SAVED STATE OF ANOTHER PROCESS OR THREAD SO THAT THE CPU CAN SWITCH EXECUTION BETWEEN THEM.

- WHEN CPU SWITCHES TO ANOTHER PROCESS, THE SYSTEM MUST **SAVE THE STATE** OF THE OLD PROCESS AND **LOAD THE SAVED STATE** FOR THE NEW PROCESS VIA A CONTEXT SWITCH

CONTEXT SWITCHING



OPERATIONS ON PROCESSES

- SYSTEM MUST PROVIDE MECHANISMS FOR:
- PROCESS CREATION,
- PROCESS TERMINATION,
- AND SO ON AS DETAILED NEXT

PROCESS CREATION

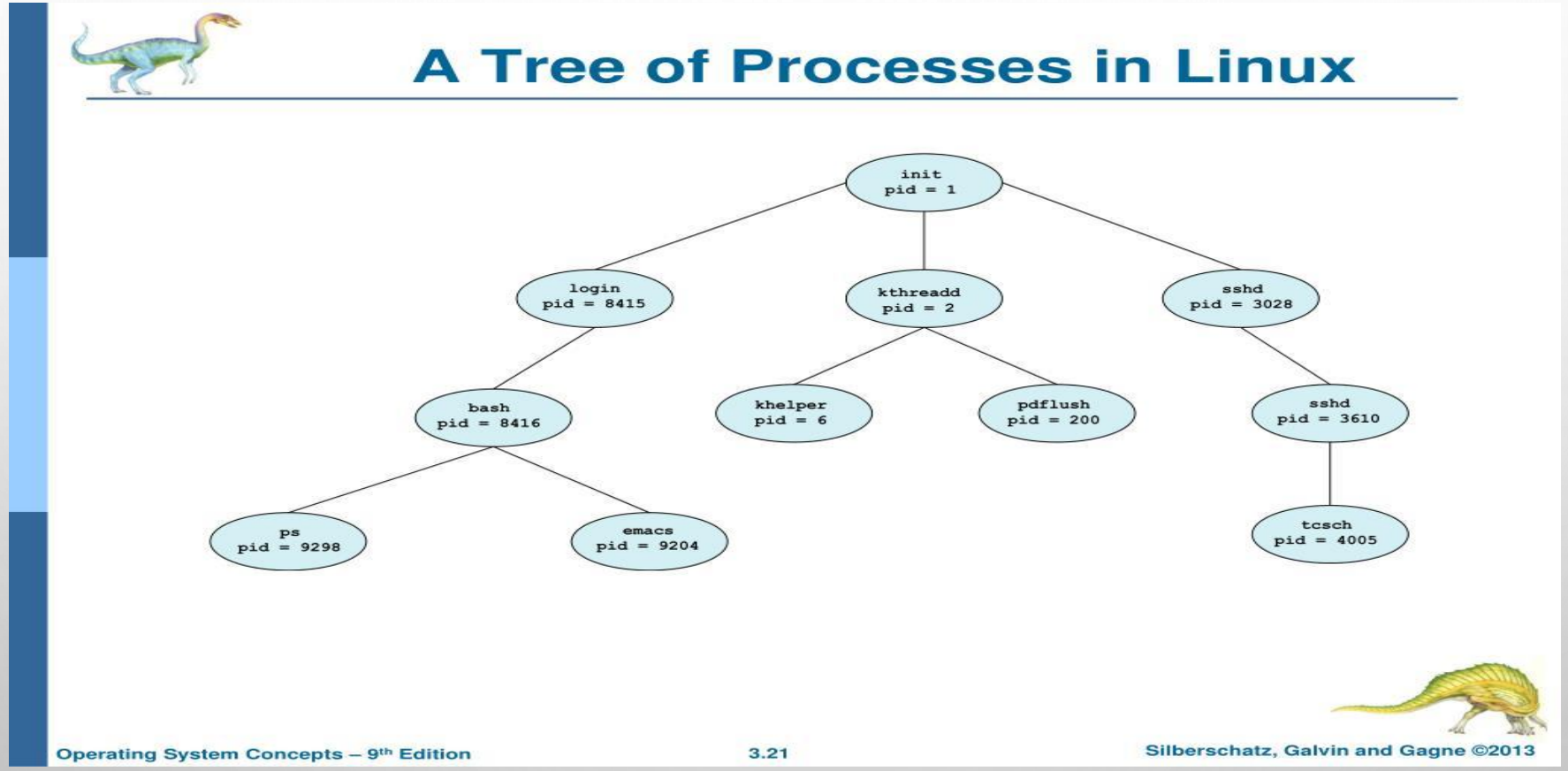
- PARENT PROCESS CREATE CHILDREN PROCESSES, WHICH, IN TURN CREATE OTHER PROCESSES, FORMING A TREE OF PROCESSES
- GENERALLY, PROCESS IDENTIFIED AND MANAGED VIA A PROCESS IDENTIFIER (PID)

RESOURCE SHARING OPTIONS

- ❑ PARENT AND CHILDREN SHARE ALL RESOURCES
- ❑ CHILDREN SHARE SUBSET OF PARENT'S RESOURCES
- ❑ PARENT AND CHILD SHARE NO RESOURCES

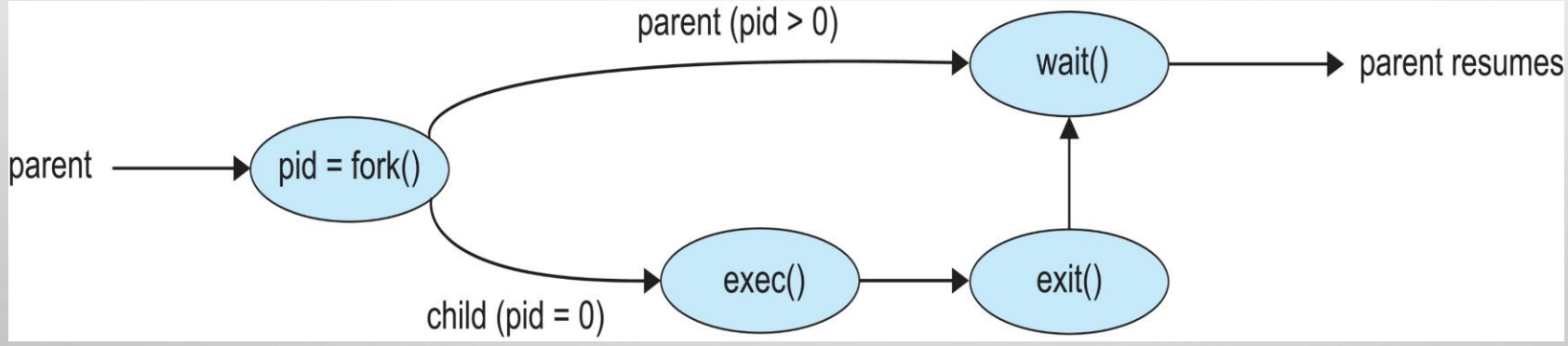
- EXECUTION OPTIONS
  - ❑ PARENT AND CHILDREN EXECUTE CONCURRENTLY
  - ❑ PARENT WAITS UNTIL CHILDREN TERMINATE

A TREE OF PROCESSES IN LINUX



PROCESS CREATION (CONT.)

- ADDRESS SPACE OPTIONS
  - ❑ CHILD DUPLICATE OF PARENT
  - ❑ CHILD HAS A PROGRAM LOADED INTO IT
- UNIX EXAMPLES
  - ❑ **FORK()**SYSTEM CALL CREATES NEW PROCESS
  - ❑ **EXEC()**SYSTEM CALL USED AFTER A FORK() TO REPLACE THE PROCESS' MEMORY SPACE WITH A NEW PROGRAM



PROCESS TERMINATION

- PROCESS EXECUTES LAST STATEMENT AND THEN ASKS THE OPERATING SYSTEM TO DELETE IT USING THE EXIT() SYSTEM CALL.
  - RETURNS STATUS DATA FROM CHILD TO PARENT (VIA WAIT())
  - PROCESS' RESOURCES ARE DEALLOCATED BY OPERATING SYSTEM
- PARENT MAY TERMINATE THE EXECUTION OF CHILDREN PROCESSES USING THE ABORT() SYSTEM CALL. SOME REASONS FOR DOING SO:
  - CHILD HAS EXCEEDED ALLOCATED RESOURCES
  - TASK ASSIGNED TO CHILD IS NO LONGER REQUIRED
  - THE PARENT IS EXITING AND THE OPERATING SYSTEMS DOES NOT ALLOW A CHILD TO CONTINUE IF ITS PARENT TERMINATES

PROCESS TERMINATION

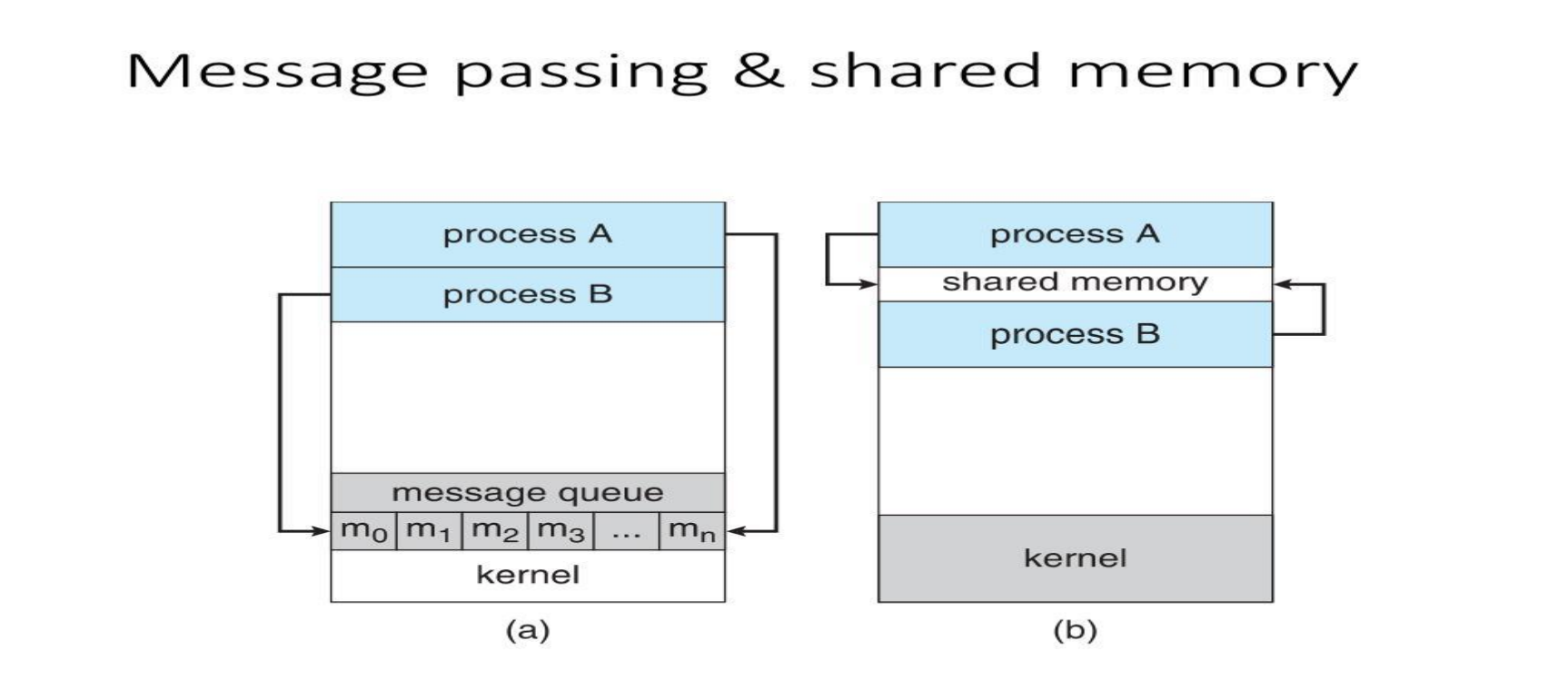
- SOME OPERATING SYSTEMS DO NOT ALLOW CHILD TO EXISTS IF ITS PARENT HAS TERMINATED. IF A PROCESS TERMINATES, THEN ALL ITS CHILDREN MUST ALSO BE TERMINATED.
  - ❑ CASCADING TERMINATION. ALL CHILDREN, GRANDCHILDREN, ETC. ARE TERMINATED.
  - ❑ THE TERMINATION IS INITIATED BY THE OPERATING SYSTEM.
- THE PARENT PROCESS MAY WAIT FOR TERMINATION OF A CHILD PROCESS BY USING THE WAIT()SYSTEM CALL. THE CALL RETURNS STATUS INFORMATION AND THE PID OF THE TERMINATED PROCESS

  PID = WAIT(&STATUS);
- IF NO PARENT WAITING (DID NOT INVOKE WAIT()) PROCESS IS A ZOMBIE
- IF PARENT TERMINATED WITHOUT INVOKING WAIT , PROCESS IS AN ORPHAN

INTERPROCESS COMMUNICATION

- PROCESSES WITHIN A SYSTEM MAY BE INDEPENDENT OR COOPERATING
- COOPERATING PROCESS CAN AFFECT OR BE AFFECTED BY OTHER PROCESSES, INCLUDING SHARING DATA
- INDEPENDENTPROCESS CANNOT AFFECT OR BE AFFECTED BY THE EXECUTION OF ANOTHER PROCESS
- REASONS FOR COOPERATING PROCESSES:
  - INFORMATION SHARING
  - COMPUTATION SPEEDUP
  - MODULARITY
- COOPERATING PROCESSES NEED INTERPROCESS COMMUNICATION (IPC)
- TWO MODELS OF IPC
  - SHARED MEMORY
  - MESSAGE PASSING

INTERPROCESS COMMUNICATION



(a) Message passing. (b) shared memory.